

Text Mining Basics

- Text mining extracts meaningful information from text.
- Includes cleaning, tokenization, and vectorization.

Corpus

- A corpus is a collection of documents.
- Used as the main input for text mining tasks.

Document-Term Matrix (DTM)

- DTM is a matrix where:
- Rows = documents
- Columns = terms
- Values = frequency or weight of terms.

Weighting Techniques

- TF (Term Frequency): counts occurrences.
- Binary: 1 if term exists, else 0.

Handling Sparsity

- Most DTMs are sparse.
- We remove rare terms to reduce dimensionality.

N-grams

- N-grams are sequences of words.
- Bigram = 2-word combinations.
- Helps capture context.

Python Implementation

- Using sklearn:
- CountVectorizer for DTM
- ngram_range for bigrams
- min_df for sparsity

DTM Example

- Documents:
- D1: 'small example'
- D2: 'small small test'
- Terms: small, example, test

DTM Matrix Illustration

- small example test
- D1 1 1 0
- D2 2 0 1
- Rows=documents, Columns=terms

What is TF-IDF?

- $\text{TF-IDF} = \text{Term Frequency} * \text{Inverse Document Frequency}$
- Highlights important words, reduces common ones.

TF (Term Frequency)

- Measures how often a term appears in a document.
- Example: 'small' appears 2 times $\rightarrow$ TF=2

IDF (Inverse Document Frequency)

- Reduces weight of common words.
- Formula: $\log(N / df)$
- N = total docs, df = docs containing term

TF-IDF Example

- If 'small' appears in all docs → low IDF
- If 'example' appears in few docs → high IDF
- TF-IDF favors rare but meaningful terms

TF-IDF in Python

- `from sklearn.feature_extraction.text import TfidfVectorizer`
- `vectorizer = TfidfVectorizer()`
- `X = vectorizer.fit_transform(text)`

Why TF-IDF is Important

- Removes noise (common words)
- Improves search, clustering, classification
- Widely used in NLP and cyber intelligence

Cosine Similarity

- Measures similarity between documents.
- Range: 0 (different) $\rightarrow$ 1 (identical)
- Based on vector angle, not magnitude.

Cosine Similarity Formula

- $\cos(\theta) = (A \cdot B) / (||A|| * ||B||)$
- A,B = document vectors (TF-IDF)

Cosine Similarity in Python

- `from sklearn.metrics.pairwise import cosine_similarity`
- `sim = cosine_similarity(X)`
- Used for document matching and clustering

Real Dataset Example

- Input: Telegram / logs / JSON messages
- Fields: text, user, date
- Goal: extract patterns and connections

Pipeline Example Code

- `from sklearn.feature_extraction.text import TfidfVectorizer`
- `from sklearn.metrics.pairwise import cosine_similarity`
- `texts = [msg1, msg2, msg3]`
- `X = TfidfVectorizer().fit_transform(texts)`
- `similarity = cosine_similarity(X)`

Text Mining Pipeline

- Raw Text → Cleaning → Tokenization → DTM
→ TF-IDF → Similarity → Insights

Use Case (Cyber / Intelligence)

- Detect similar messages
- Link users to topics
- Identify repeated patterns
- Build graph (Neo4j) from similarities